

Veterinary Clinic Management System



Debrecen SZC Mechwart András Technical School of Mechanical Engineering and
Information Technology

Prepared by:
Balogh Bence
Csomós János
Ráczempek Lajos

Software Developer and Tester Training Program

Debrecen, 2025-2026 academic year

Table of Contents

1. Introduction	3
1.1 Topic Overview	3
1.2 Rationale for Choosing the Topic	3
1.3 Purpose and Target Audience of the System	4
1.4 Main Functions	4
2. Requirements Analysis	5
2.1 System Requirements	5
3. Development Environment	6
3.1 Hardware and Software Requirements	6
4. Technologies Used	7
5. System Architecture	8
6. Backend Development	9
7. Data Models	10
8. REST API	14
9. User Authentication and Security	16
10. Administration Interface	17
11. Frontend Development and User Interface	18
12. Using the Application	25
13. Installation and Running Guide	26
14. Test Documentation	27
15. Possible Future Improvements	29
16. Summary	30

1. Introduction

1.1 Topic Overview

The final project is a web-based veterinary clinic management system designed to support the everyday administration of a veterinary practice. Its purpose is to provide a single interface for managing owners and their pets, appointments, veterinarians, vaccinations, medical history and prescriptions.

The backend is built with Django and Django REST Framework, data is stored in an SQLite database, and the frontend accesses the required resources through a REST API. The system separates data management from presentation, allowing the two layers to be developed independently.

1.2 Rationale for Choosing the Topic

Veterinary practices need to keep track of a large amount of interconnected information at the same time. A pet may be associated with an owner, appointments, examinations, vaccinations and medication treatments. Managing these on paper or across several separate systems can be difficult to oversee. The project therefore presents a unified, easy-to-use solution that manages clinic processes using a structured data model.

From a professional perspective, the final project also has an appropriate level of complexity: it includes database management, a REST API, authentication, authorization, image upload, frontend-backend communication and an administration interface.

1.3 Purpose and Target Audience of the System

The primary purpose of the system is to simplify clinical administration, while the application is not intended exclusively for administrators. In addition to clinic staff, private pet owners can also register, record their own pets and request appointments at the veterinary practice.

- • pet owners who manage their own account and pet profiles;
 - • administrative staff of veterinary practices and smaller clinics;
 - • veterinarians who need to review previous treatments and vaccinations;
 - • users with administrator privileges;
-
- people testing or evaluating the development project.

1.4 Main Functions

- • User registration and JWT-based login.
- • Recording and managing the user's own pets.
- • Requesting an appointment; new bookings are pending by default.
- • Administrator review and status management of bookings.
- • Managing owners, veterinarians, treatments, vaccinations and prescriptions.
- • Image upload, Django administration and a responsive React user interface.
- • Token refresh and logout, with permission-based data access.
-
-
-

2. Requirements Analysis

2.1 System Requirements

Component	Minimum requirement	Recommended
Operating system	Windows 10 64-bit	Windows 11 64-bit
Processor	2 cores, 2.0 GHz	4 cores or better
Memory	4 GB RAM	8 GB RAM or more
Storage	2 GB free space	5 GB or more
Display	1280×720	1920×1080
Browser	Current Chrome / Edge / Firefox	Latest stable version
Development runtime	Python 3, Node.js, npm, Git	Current stable/LTS releases

When designing the system, the primary considerations were a clear user interface, structured data storage, secure access and future extensibility. The client and server operate as separate layers, so the user interface and backend can be developed and tested independently.

Using the system requires a modern web browser, a stable local or network connection, and proper installation of the server-side and client-side environments. Protected data is accessible only after login.

- Clear, consistent, Hungarian-language user interface.
- Authenticated access to protected clinical data.
- Modular structure with the backend and frontend able to run separately.
- The system can be run in both Windows and macOS development environments.
- Easy-to-install development environment.

3. Development Environment

The project was created using a client-server architecture. The backend runs in a Python and Django environment, while the frontend uses React and Vite. Git can be used for source-code version control, and Visual Studio Code or another modern code editor is suitable for development. Windows is the primary demonstration environment, although the system can also be run on macOS.

Area	Tool / technology	Role
Backend	Python	Server-side programming language
Backend	Django	Web framework and ORM
API	Django REST Framework	REST endpoints and serialization
Database	SQLite	Development database
Security	Simple JWT	Access and refresh token handling
Media	Pillow	Image upload and processing
Admin	Django Admin	Built-in administration
Frontend	React 19 / React Router	User interface and client-side routing
Frontend runtime	Vite / Node.js / npm	Development server and package management

3.1 Hardware and Software Requirements

The user-facing application itself does not require significant computing power, so it can be used on an average office or home computer. In a development environment, the backend and frontend run simultaneously; therefore, at least 4 GB of RAM and a few gigabytes of free storage are required. The table above summarizes the minimum Windows-based configuration.

4. Technologies Used

4.1 Django and Django REST Framework

Django provides the data models, ORM, server-side configuration and built-in administration interface. Django REST Framework makes the data models available through REST endpoints. ModelViewSet and DefaultRouter simplify CRUD operations when defining the endpoints.

4.2 SQLite

The project uses an SQLite database. For development and examination projects, its advantage is that data can be stored in a single file without installing a separate database server. A later database migration can also be handled through the Django ORM.

4.3 JWT and Simple JWT

Authentication is performed using JWT tokens. After a successful login, an access token and a refresh token are created. The access token is used to access protected endpoints, while the refresh token is used to request a new access token.

4.4 Pillow and Image Upload

A photograph can also be assigned to each pet. The backend uses an ImageField, while image handling is supported by the Pillow library.

4.5 React, Vite and Client-Side Libraries

The user interface is based on React 19. Vite provides the development server and build process, react-router-dom is used for routing, and lucide-react provides icons. Components are organized by function into pages, elements and services.

5. System Architecture

The application can be divided into three main layers: user interface, REST API and database. The frontend sends HTTP requests to the backend. The backend performs authentication, business logic and database operations, then returns a JSON response.

Frontend → REST API → Django models / ORM → SQLite database

Separating the layers is beneficial because changes to the user interface do not require a complete redesign of the database, and the API may also be used by other clients.

5.1 Example Data Flow - Appointment Booking

1. The user selects the owner, pet, veterinarian, date and time.
2. The frontend sends the booking data to the appropriate backend API endpoint.
3. The backend verifies authentication and processes the incoming data.
4. A record belonging to the Appointment model is created in the database.
5. Based on the response, the frontend provides feedback and the booking can then appear in the list.

6. Backend Development

The backend is responsible for managing and storing the data required for the operation of the veterinary practice and making it available to the frontend through a REST API. Separate models were created for the main business entities.

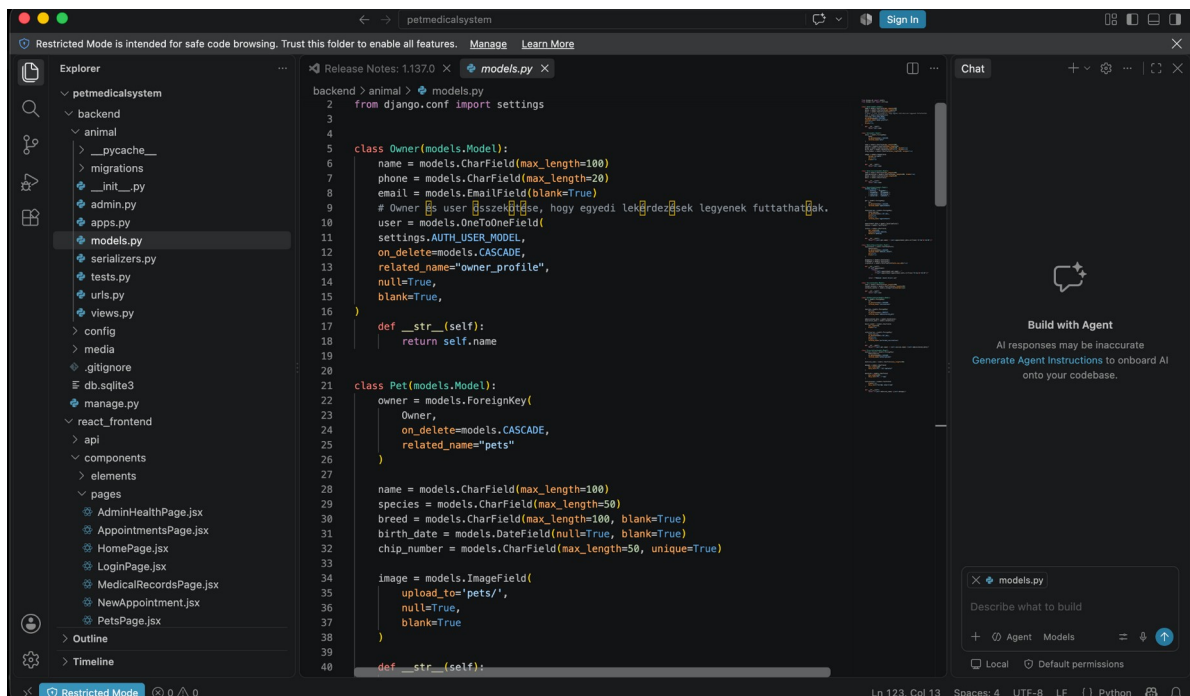
6.1 Backend Modules

- Owner - owners
- Pet - pets
- Veterinarian - veterinarians
- Appointment - appointments
- MedicalRecord - medical records
- Vaccine - vaccine types
- PetVaccination - vaccination events
- Prescription - prescriptions

6.2 Relationships

Foreign Key, One-to-One and, where necessary, Many-to-Many relationships can be used between the models. For example, one Owner record may have several Pets, while an appointment is linked to a pet and a veterinarian.

6.3 Backend Code Example - Owner and Pet Models



```
1 from django.conf import settings
2
3
4
5 class Owner(models.Model):
6     name = models.CharField(max_length=100)
7     phone = models.CharField(max_length=20)
8     email = models.EmailField(blank=True)
9     # Owner is user, hogy egyedi lekérdezések legyenek futtathatók.
10    user = models.OneToOneField(
11        settings.AUTH_USER_MODEL,
12        on_delete=models.CASCADE,
13        related_name="owner_profile",
14        null=True,
15        blank=True,
16    )
17
18    def __str__(self):
19        return self.name
20
21
22 class Pet(models.Model):
23     owner = models.ForeignKey(
24         Owner,
25         on_delete=models.CASCADE,
26         related_name="pets"
27     )
28
29     name = models.CharField(max_length=100)
30     species = models.CharField(max_length=50)
31     breed = models.CharField(max_length=100, blank=True)
32     birth_date = models.DateField(null=True, blank=True)
33     chip_number = models.CharField(max_length=50, unique=True)
34
35     image = models.ImageField(
36         upload_to='pets/',
37         null=True,
38         blank=True
39     )
40
41    def __str__(self):
```

Figure - Owner and Pet models in models.py

The Owner model is connected to the Django user account through a OneToOneField. The Pet model uses a ForeignKey to connect each pet to its owner, allowing one owner to have multiple pets.

7. Data Models

7.1 Owner - owner

The Owner model stores the basic details of pet owners. Its main fields are name, phone number and email address. One owner may have several pets, so there is a one-to-many relationship between Owner and Pet.

Field	Meaning
name	Owner name
phone	Phone number
email	Email address

7.2 Pet - pet

The Pet model is one of the central elements of the clinical system. Owner information, appointments, treatment history, vaccinations and prescriptions are linked to the pet.

Field	Meaning
name	Pet name
species	Species
breed	Breed
birth_date	Date of birth
chip_number	Chip number
owner	Owner relationship
image	Photograph

7.3 Veterinarian - veterinarian

The Veterinarian model stores veterinarian information. It contains name, specialization, phone number and email address. Veterinarians can be assigned to appointments and vaccination events.

Field	Meaning
name	Veterinarian name
specialization	Specialization
phone	Phone number
email	Email address

7.4 Appointment - appointment

The Appointment model manages appointments related to pets. It contains the pet concerned, date and time, reason for the visit, assigned veterinarian and status.

Status	Meaning
scheduled	scheduled
completed	completed
cancelled	cancelled

Based on the frontend screenshot, the booking process allows the user to select an owner, pet and veterinarian, as well as enter a date, time and reason for the appointment.

7.5 MedicalRecord - medical record

7.4.1 Backend Code Example - Appointment Model

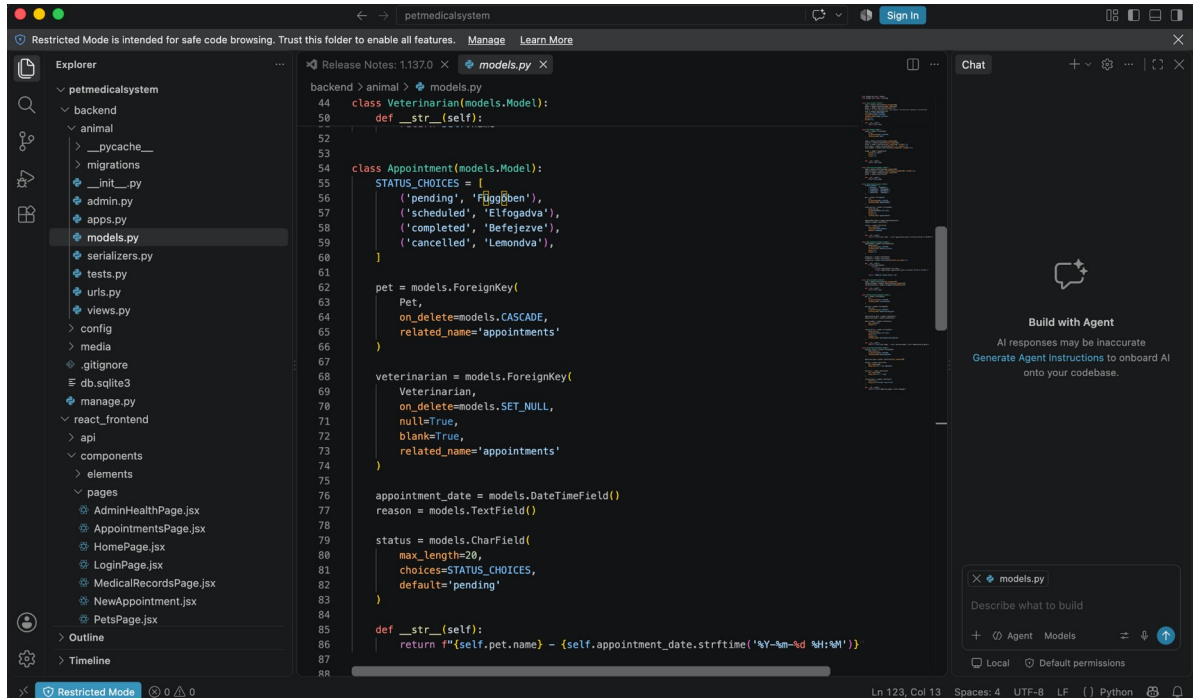


Figure - Appointment model and booking statuses

The Appointment model connects a pet with an optional veterinarian and a specific date and reason. A new booking receives the pending status by default; it can later be accepted (scheduled), completed or cancelled.

The MedicalRecord model stores examinations and diagnoses related to pets. An entry contains the pet concerned, the related appointment, diagnosis, treatment and creation date. The relationship with the appointment helps retrieve the specific examination event.

7.6 Vaccine and PetVaccination

The vaccination system separates two different concepts. Vaccine represents a vaccine type, while PetVaccination represents a vaccination event administered to a specific animal. This separation reduces duplicate data and allows the same vaccine type to be recorded for multiple animals.

- vaccine type
- date administered
- expiration date
- batch/lot number
- veterinarian administering the vaccine

7.7 Prescription - prescriptions

The Prescription model stores medication prescribed as part of a medical record. Each prescription is linked to a MedicalRecord, so multiple prescriptions may belong to one examination or treatment.

The main fields are medicine_name, dosage, duration and instructions.

Entity	Related area
Owner	Pet
Pet	Owner, Appointment, MedicalRecord, PetVaccination, Prescription
Veterinarian	Appointment, PetVaccination
Appointment	Pet, Veterinarian, MedicalRecord
Vaccine	PetVaccination
PetVaccination	Pet, Vaccine, Veterinarian
Prescription	Pet

The pet is at the center of the relational structure. This is logical because the vast majority of clinical events can be linked to a specific animal.

7.9 Database Relationship Diagram

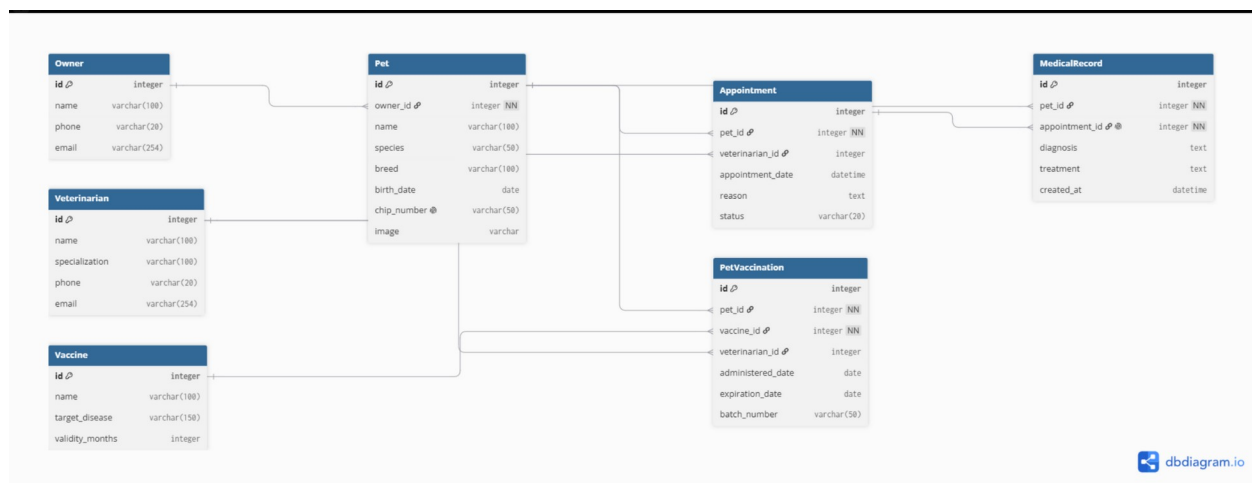


Figure - Relationship diagram of the application's main database entities

The diagram provides a visual overview of the main database entities and their relationships. It complements the detailed model descriptions above and makes the links between owners, pets, appointments, veterinarians, medical records and vaccination data easier to review.

8. REST API

One of the most important parts of the backend is the REST API. Django REST Framework ModelViewSet and DefaultRouter solutions provide resource management.

Endpoint	Resource
/api/owners/	Owners
/api/pets/	Pets
/api/appointments/	Appointments
/api/medical-records/	Medical records
/api/veterinarians/	Veterinarians
/api/vaccines/	Vaccines
/api/pet-vaccinations/	Vaccination events
/api/prescriptions/	Prescriptions

8.1 REST Approach

The REST API allows the frontend to retrieve, create or modify data in a consistent way. The specific HTTP methods and endpoint configurations can be verified in the project source code, while the API endpoints ensure that individual clinical resources are managed separately.

8.2 Example API Workflows

Retrieving Pets

The client uses the `/api/pets/` resource to access pet data. Based on the response, the frontend can display the animals in cards or a list.

Managing Appointments

The `/api/appointments/` resource is used to manage appointment bookings. Appointment status is also visible in the user interface and can be modified by users with an administrative role.

Vaccination Data

The `/api/vaccines/` endpoint manages vaccine types, while `/api/pet-vaccinations/` manages individual vaccination events. Together, these two endpoints enable structured recording of clinical vaccination history.

Prescriptions

The `/api/prescriptions/` endpoint supports providing prescriptions associated with an animal to the frontend.

9. User Authentication and Security

The backend uses JWT-based authentication with Django REST Framework Simple JWT. An access token and refresh token are generated at login. Protected API endpoints use `IsAuthenticated` permission checks.

Endpoint	Task
POST /api/login/	Login and token creation
POST /api/token/refresh/	Refresh access token

A logout function was created to end the session by invalidating the refresh token.

9.1 Unauthorized Access

Without authentication, protected endpoints may return a response such as: `{"detail": "Authentication credentials were not provided."}`

9.2 Security Considerations

- protected API endpoints
- JWT token-based session
- permission checks
- built-in security mechanisms of the Django framework

9.3 Backend Code Example - Appointment Permissions

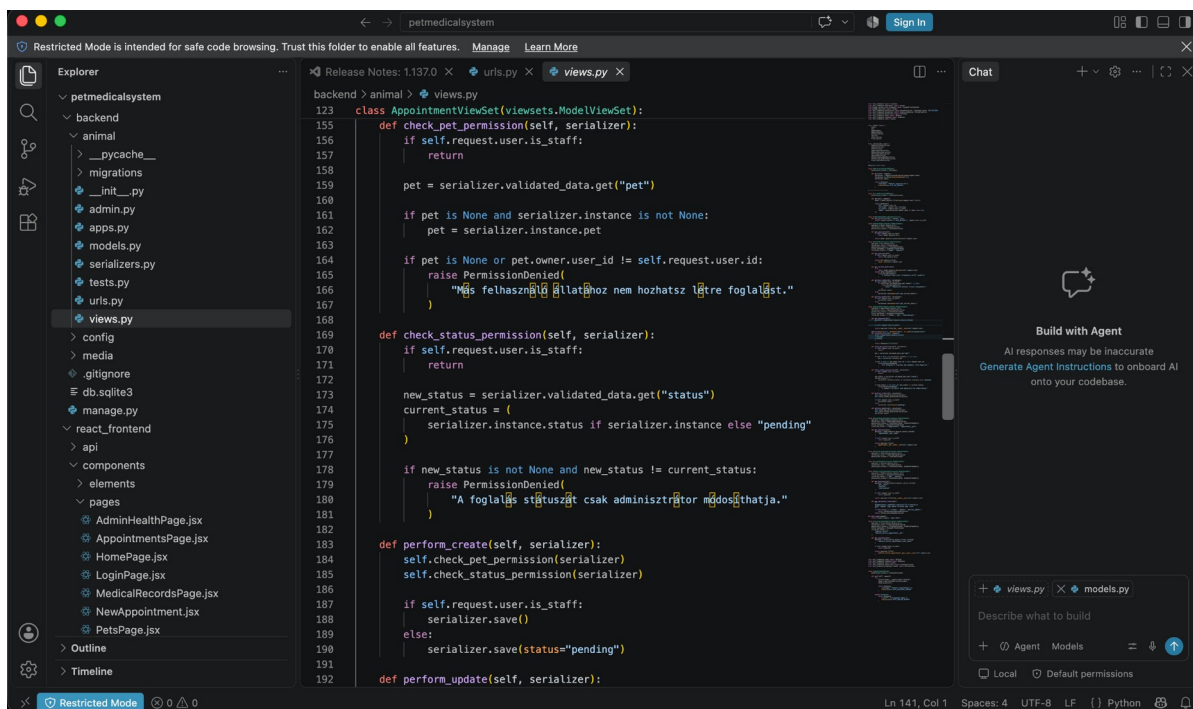


Figure - Permission and booking logic in AppointmentViewSet

The backend verifies that a regular user can create a booking only for their own pet. Bookings created by non-staff users are saved with pending status, while changing the booking status is restricted to administrators.

9.4 Backend Code Example - REST API and JWT Endpoints

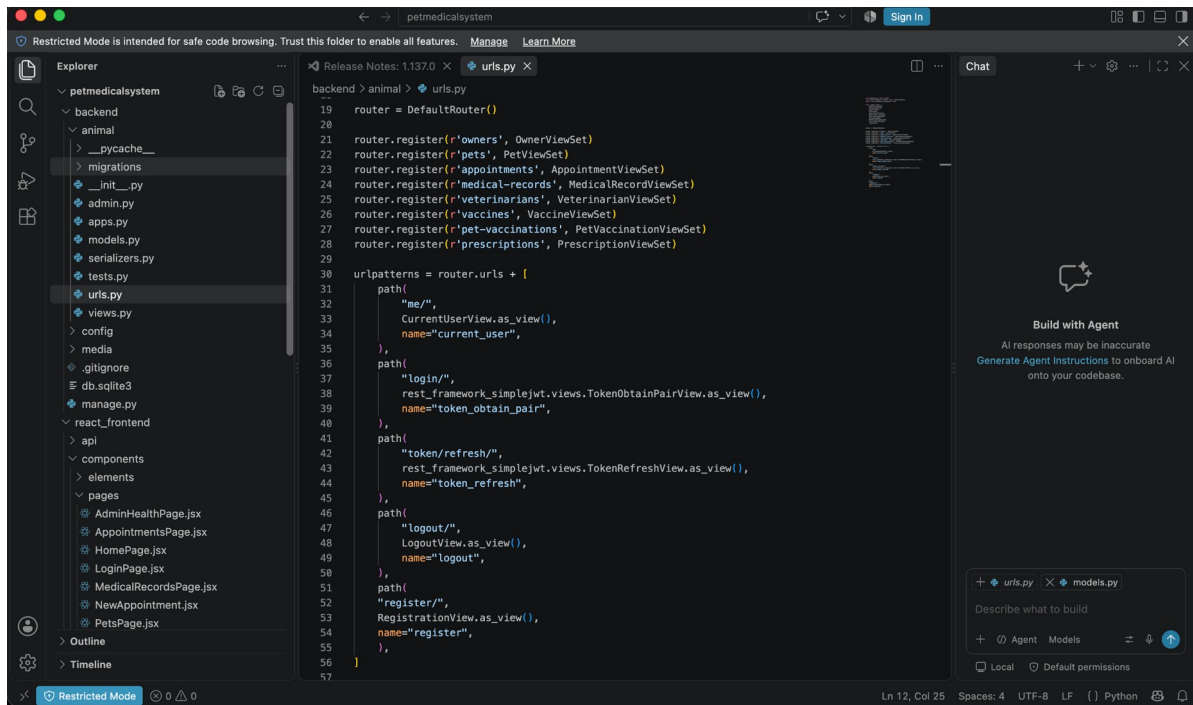


Figure - DefaultRouter registrations and authentication endpoints

DefaultRouter registers the main REST resources, while separate endpoints handle the current user, login, token refresh, logout and registration. This structure keeps resource routing and JWT-based authentication clearly separated.

10. Administration Interface

The system also uses Django's built-in administration interface. From here, the administrator can manage owners, pets, veterinarians, appointments, medical records and vaccinations.

Administration is especially important for managing bookings because the administrator can review and modify appointment statuses.

10.1 Benefits of Administration

- Fast data verification during development and testing.
- Central management of model records.
- Monitoring appointments and statuses.
- Administrative access independent of the frontend.

Django Admin does not replace the end-user interface; instead, it serves as a supplementary management tool with elevated privileges.

11. Frontend Development and User Interface

The client-side application is based on React 19 and Vite. The frontend presents the data provided by the Django REST API through an easy-to-use, Hungarian-language web interface. The code is divided into page, element and service components so that individual functions remain clearly separated.

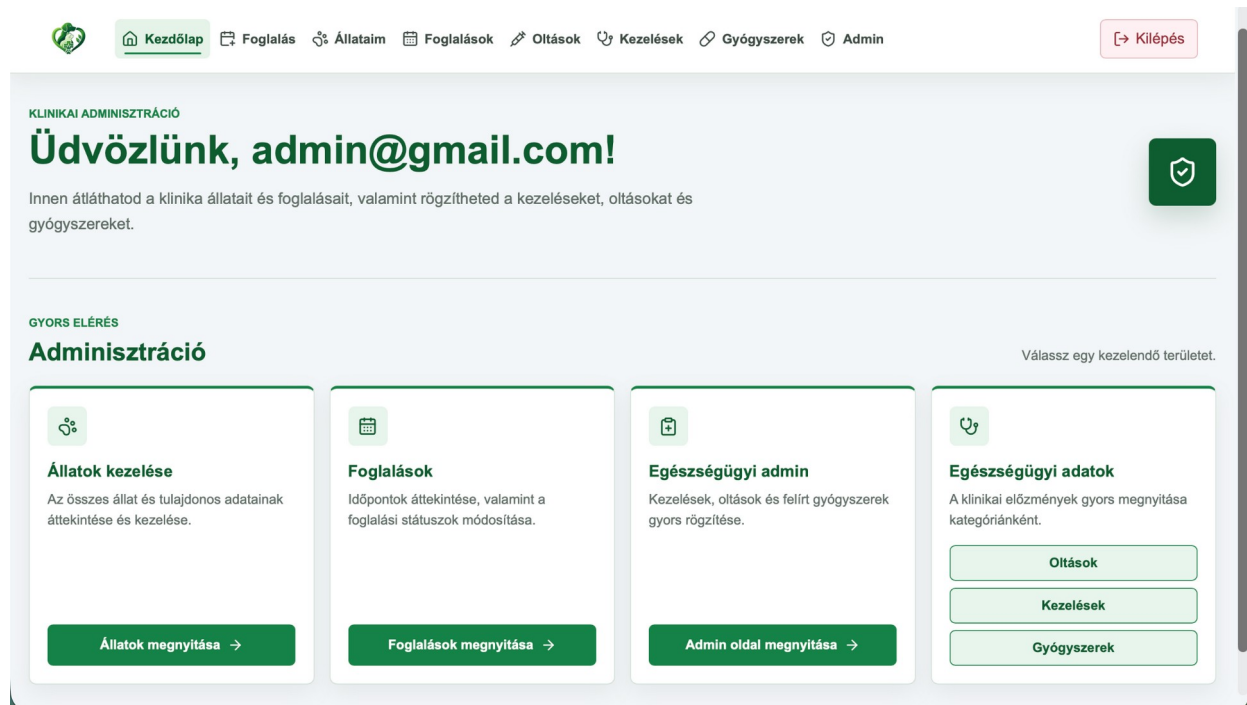


Figure 1 - Administrator home page

The top navigation provides direct access to Home, Booking, My Pets, Appointments, Vaccinations, Treatments, Medications and - with the appropriate permissions - the Admin page. Quick links on the administrator home page support the most important clinical operations.

11.1 Frontend Technologies and Project Structure

The main frontend dependencies are react, react-dom, react-router-dom and lucide-react. Vite provides the development server, fast reload and build process. CSS files are separated by functional pages; for example, separate style sheets are used for login, appointments, pets and the administration interface.

The most important project directories are: src - application entry point and main component; components/pages - page components; components/elements - reusable forms; components/services - authentication helper functions; api - functions communicating with the backend; styles - CSS style sheets; public - logo and web icons.

11.2 Routing and Protected Routes

Routing is handled by react-router-dom. The /login and /register pages are publicly accessible, while the other important pages are available through the ProtectedRoute component. If there is no valid access token, the system redirects the user to the login page. Non-existent routes also redirect to the appropriate home page or login.

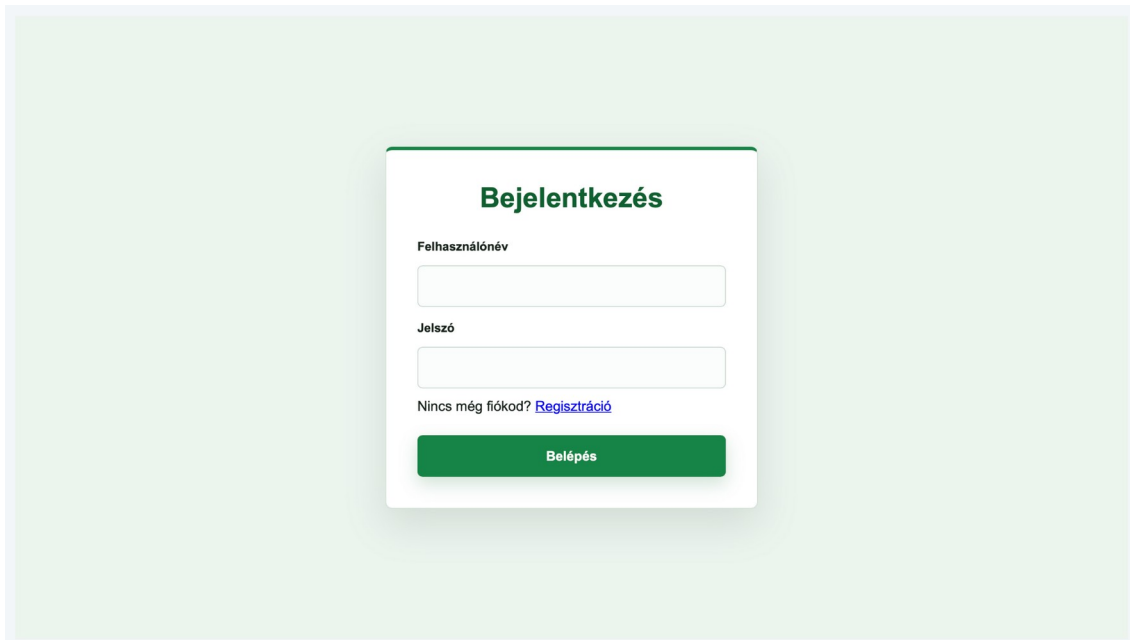


Figure 2 - Login interface

The login form asks for a username and password. After successful authentication, the user is taken to the home page; in case of an error, the interface displays a text message. Registration is directly accessible from the login page.

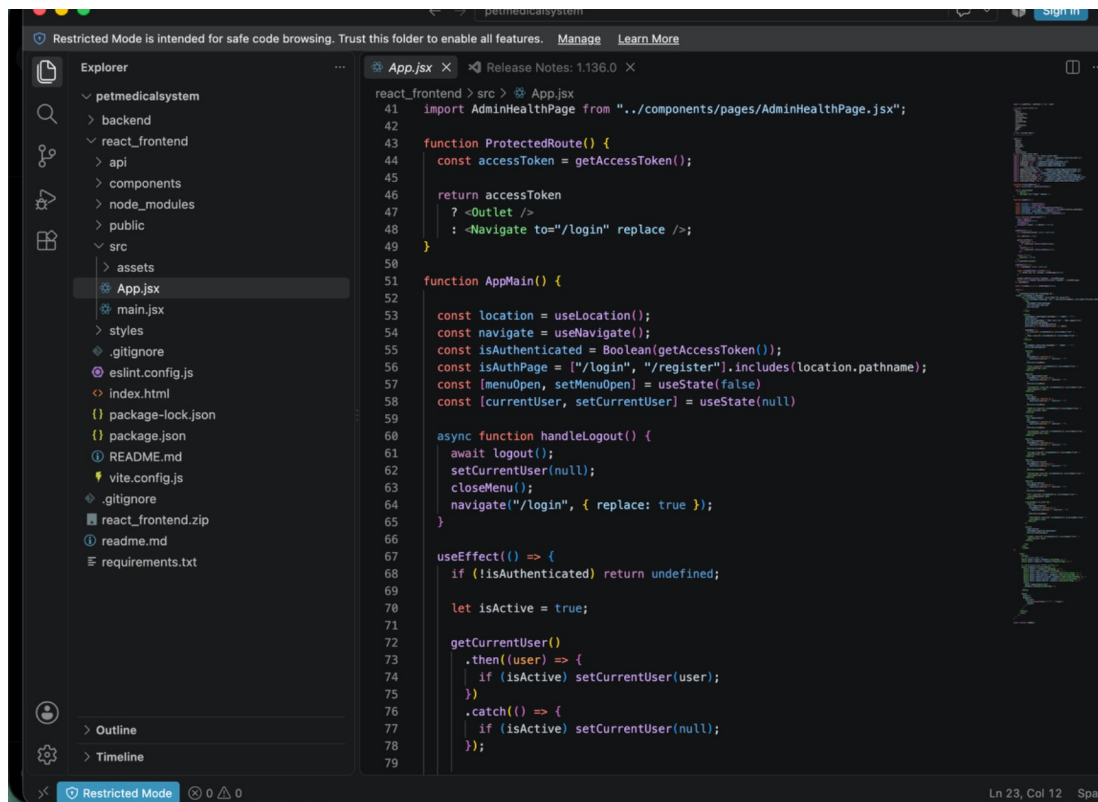


Figure 5 - Protected route and authentication in App.jsx

The ProtectedRoute component allows access to protected pages based on the presence of an access token. If the user is not logged in, the system redirects them to the login page.

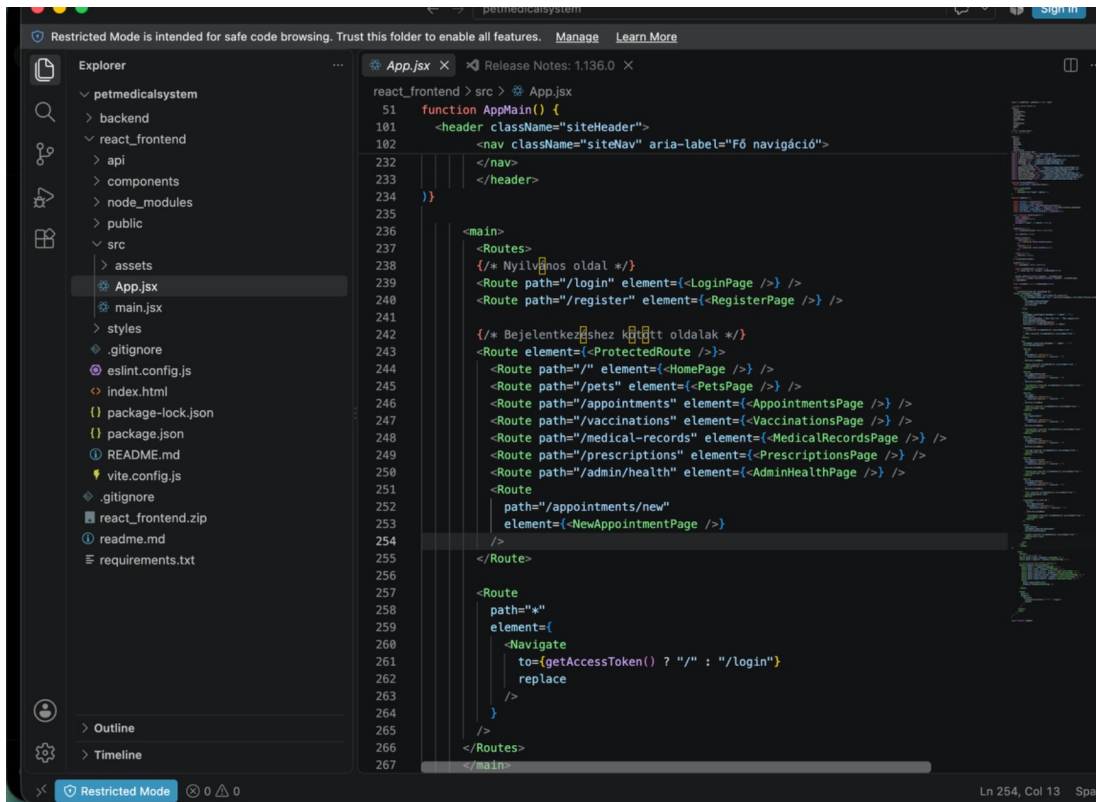


Figure 6 - React Router routes in App.jsx

The route configuration in App.jsx separates public pages from pages requiring authentication. Protected routes are accessed through a unified application layout.

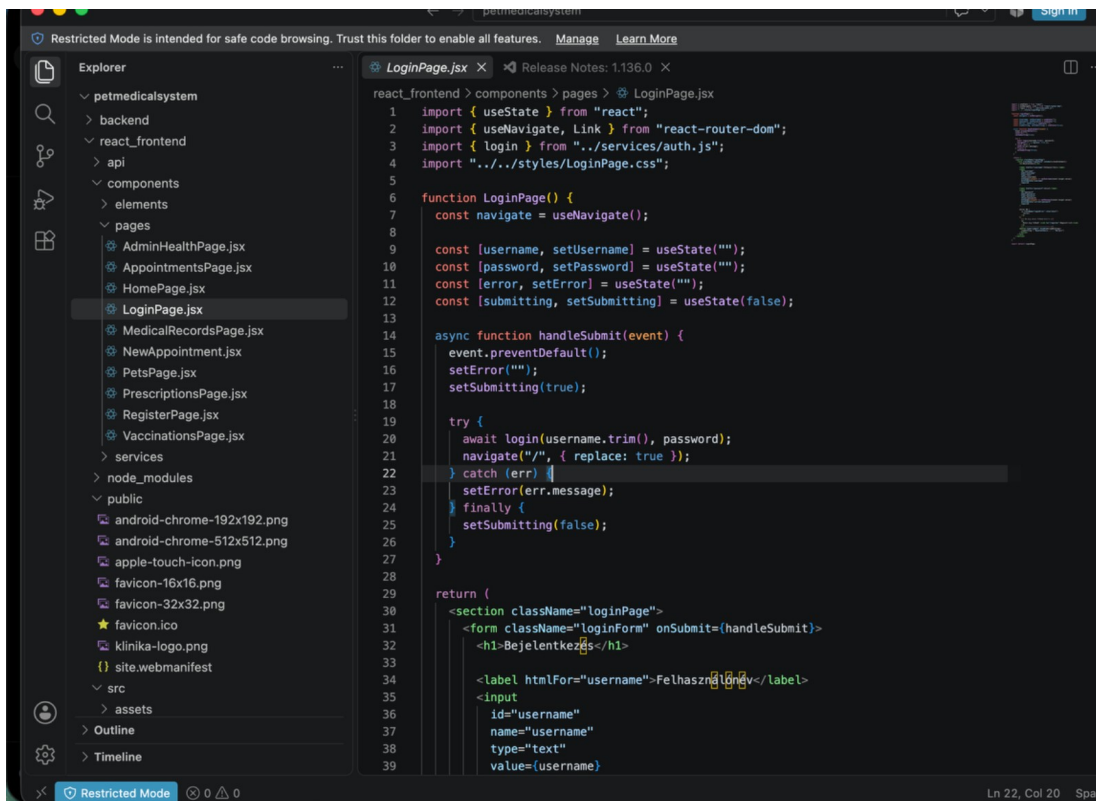


Figure 7 - Login state and error handling in LoginPage.jsx

The login component uses state variables to manage the username, password, submission state and any error message. After successful authentication, the user is taken to the home page.

11.3 JWT Handling and API Communication

The frontend's central API layer uses the base URL `http://127.0.0.1:8000/api/`. A Bearer access token can be sent in the Authorization header of requests. If the server returns a 401 response, the client attempts to refresh the token using the refresh token and then repeats the original request. If refreshing fails, the tokens are deleted and the user is returned to the login page.

Client-side token storage and login/logout operations are handled by a separate auth service. This reduces repeated code and provides consistent authentication behavior throughout the frontend.

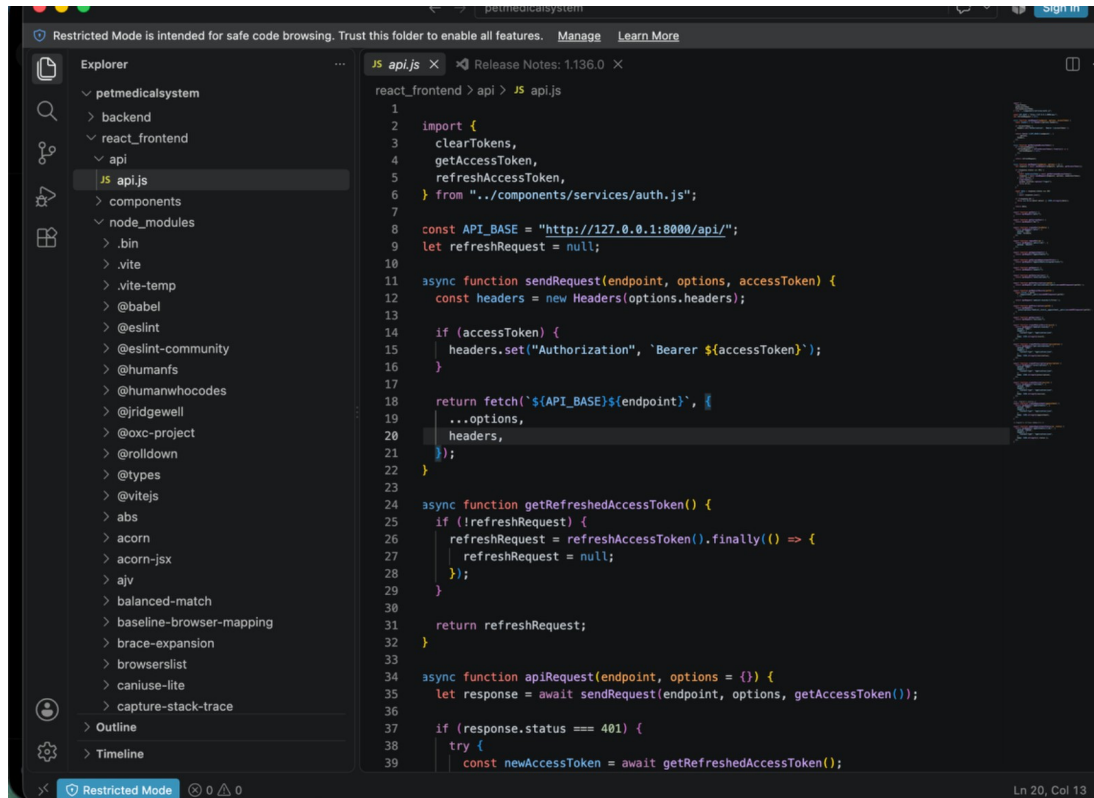
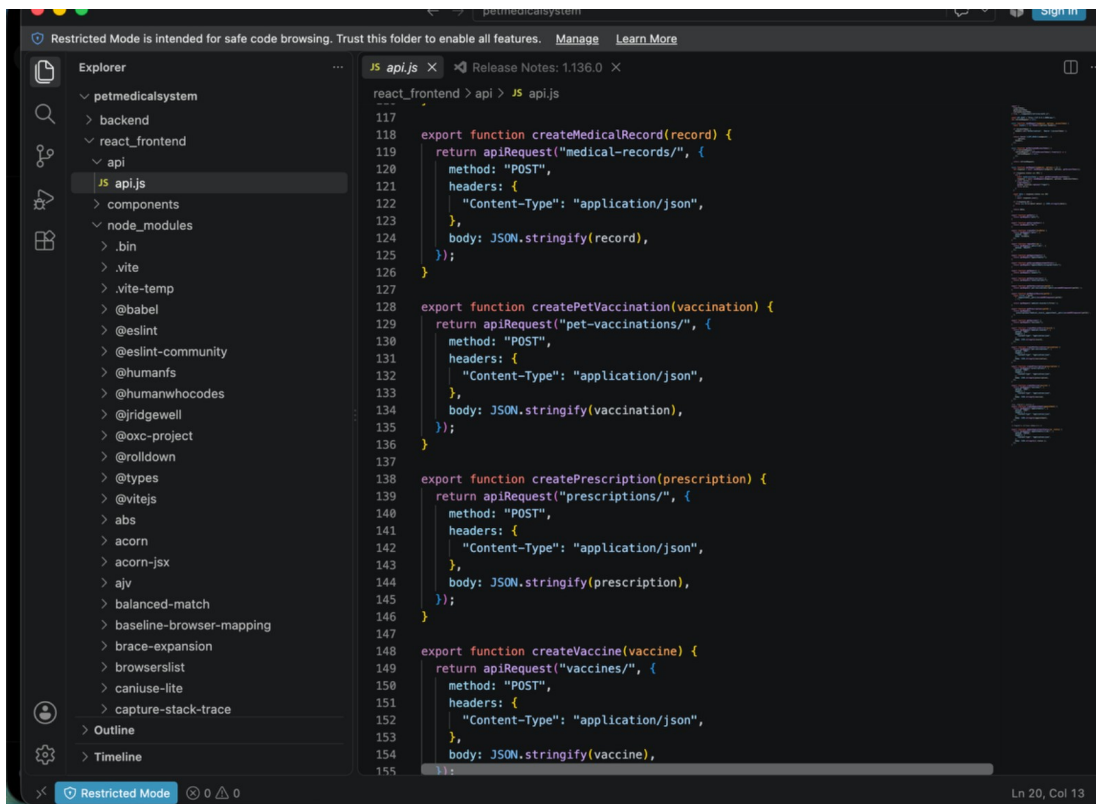


Figure 8 - JWT token and API request handling in api.js

The central API helper function automatically adds the Bearer token to requests. If the access token has expired, the client attempts to refresh it and then resends the original request.

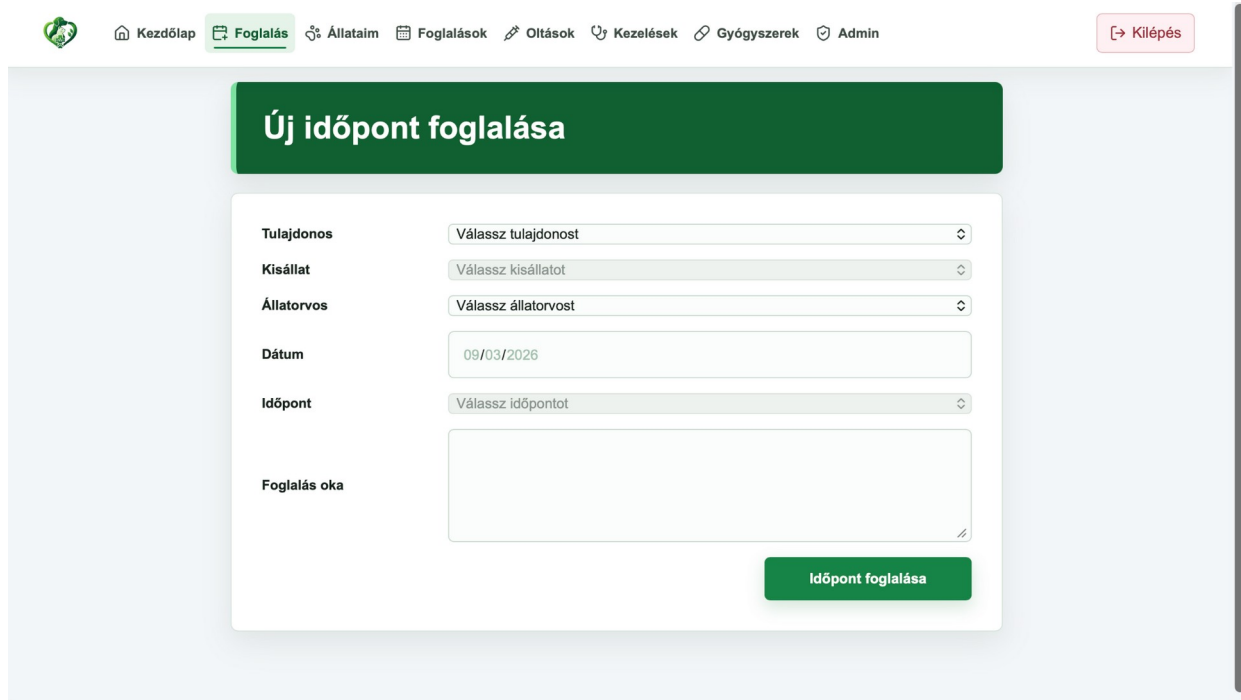


```
react_frontend > api > JS api.js
117
118 export function createMedicalRecord(record) {
119   return apiRequest("medical-records/", {
120     method: "POST",
121     headers: {
122       "Content-Type": "application/json",
123     },
124     body: JSON.stringify(record),
125   });
126 }
127
128 export function createPetVaccination(vaccination) {
129   return apiRequest("pet-vaccinations/", {
130     method: "POST",
131     headers: {
132       "Content-Type": "application/json",
133     },
134     body: JSON.stringify(vaccination),
135   });
136 }
137
138 export function createPrescription(prescription) {
139   return apiRequest("prescriptions/", {
140     method: "POST",
141     headers: {
142       "Content-Type": "application/json",
143     },
144     body: JSON.stringify(prescription),
145   });
146 }
147
148 export function createVaccine(vaccine) {
149   return apiRequest("vaccines/", {
150     method: "POST",
151     headers: {
152       "Content-Type": "application/json",
153     },
154     body: JSON.stringify(vaccine),
155   });
156 }
```

Figure 9 - Data-creation POST operations in api.js

api.js provides separate functions for creating medical data. Using the shared apiRequest function standardizes the handling of POST requests sent to the server.

11.4 Appointment Booking



The screenshot shows a web application interface for booking a new appointment. The top navigation bar includes links for Kezdőlap, Foglалás (highlighted), Állataim, Foglалások, Оltások, Kezelések, Gyógyszerek, and Admin, along with a Kіlépés button. The main heading is 'Új időpont foglalása'. The form contains several fields: 'Tulajdonos' (Owner) with a dropdown 'Válassz tulajdonost', 'Kisállat' (Pet) with a dropdown 'Válassz kisállatot', 'Állatorvos' (Veterinarian) with a dropdown 'Válassz állatorvost', 'Dátum' (Date) with a date input '09/03/2026', 'Időpont' (Appointment time) with a dropdown 'Válassz időpontot', and 'Foglalás oka' (Reason for booking) with a text area. A green 'Időpont foglalása' button is at the bottom right.

Figure 3 - Booking a new appointment

The new appointment form manages owner, pet, veterinarian, date, time and reason for booking. Related fields depend on each other; for example, selecting a pet becomes meaningful only after an owner has been selected. Before saving, the client validates the required data and then sends it to the backend in an API request.

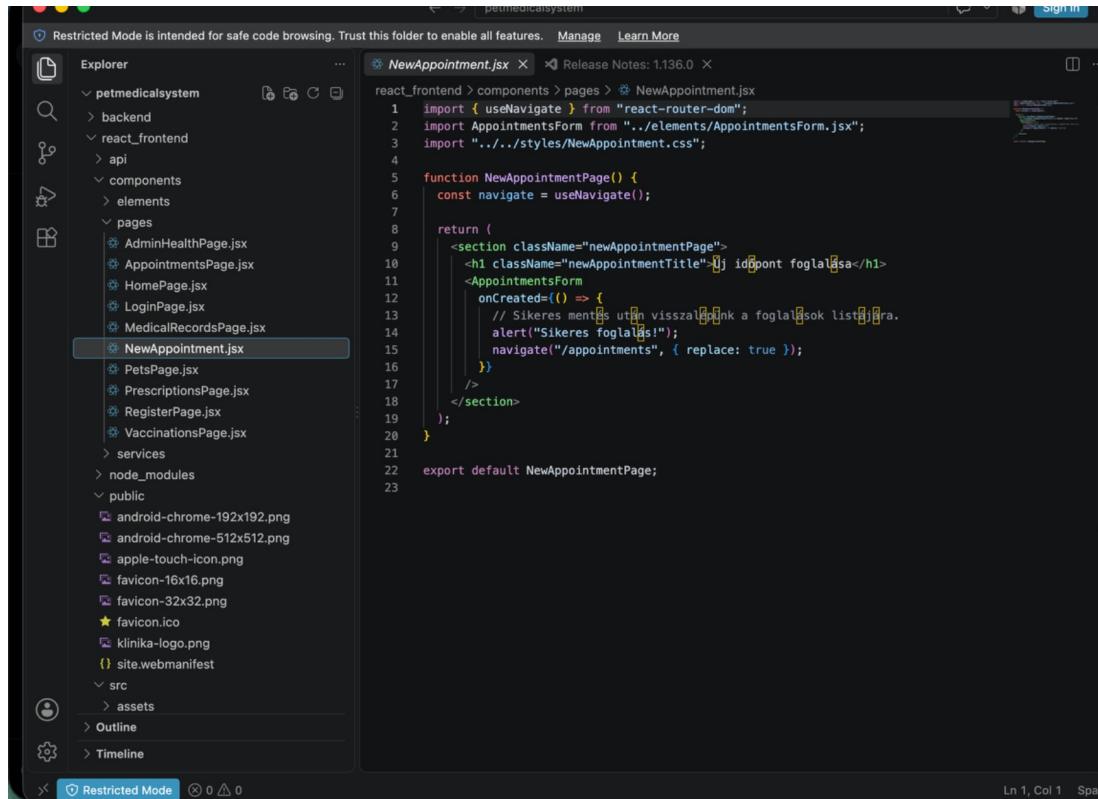


Figure 10 - New appointment page component

The NewAppointment component uses the AppointmentsForm form. After successful creation, it provides feedback to the user and then navigates to the appointments page.

11.5 Administration Frontend

The screenshot shows the 'Egészségügyi adminisztráció' (Medical Administration) interface. At the top, there is a navigation bar with icons and labels for 'Kezdőlap', 'Foglalás', 'Állataim', 'Foglalások', 'Oltások', 'Kezelések', 'Gyógyszerek', and 'Admin'. A 'Kilépés' (Logout) button is located in the top right corner. Below the navigation bar, there is a dark green header with the title 'Egészségügyi adminisztráció'. Underneath this header, there are three tabs: 'Kezelés' (selected), 'Oltás', and 'Gyógyszer'. The main content area is titled 'Új orvosi kezelés' (New medical treatment). It contains four form fields: 'Kisállat' (Pet) with a dropdown menu labeled 'Válassz kisállatot', 'Foglalás' (Appointment) with a dropdown menu labeled 'Válassz foglalást', 'Diagnózis' (Diagnosis) with a large text area, and 'Kezelés' (Treatment) with a large text area. A green button labeled 'Kezelés mentése' (Save treatment) is located at the bottom right of the form.

Figure 4 - Medical administration

The administration page combines three main areas: treatment, vaccination and medication. The interface uses separate form components for recording medical treatment, vaccination events and prescriptions. The Admin menu item appears only if the current user has `is_staff` permission.

11.6 State Management and User Feedback

The pages use the React `useState` and `useEffect` hooks to manage form data, retrieved lists, loading state and errors. Results of asynchronous API calls are saved in component state and the interface is updated accordingly. Separate feedback can be displayed while loading and in case of errors.

The navigation is responsive: on smaller screens, an expandable/collapsible menu is used and can also be closed with the Escape key. `lucide-react` icons visually support identification of individual functions.

11.7 Responsive Display

The user interface is responsive, so the application layout automatically adapts to the available screen size. On smaller displays, content elements are arranged in a single column and navigation changes to a compact hamburger menu. The mobile layout ensures that the application's main functions remain easy to access and use on smaller screens.

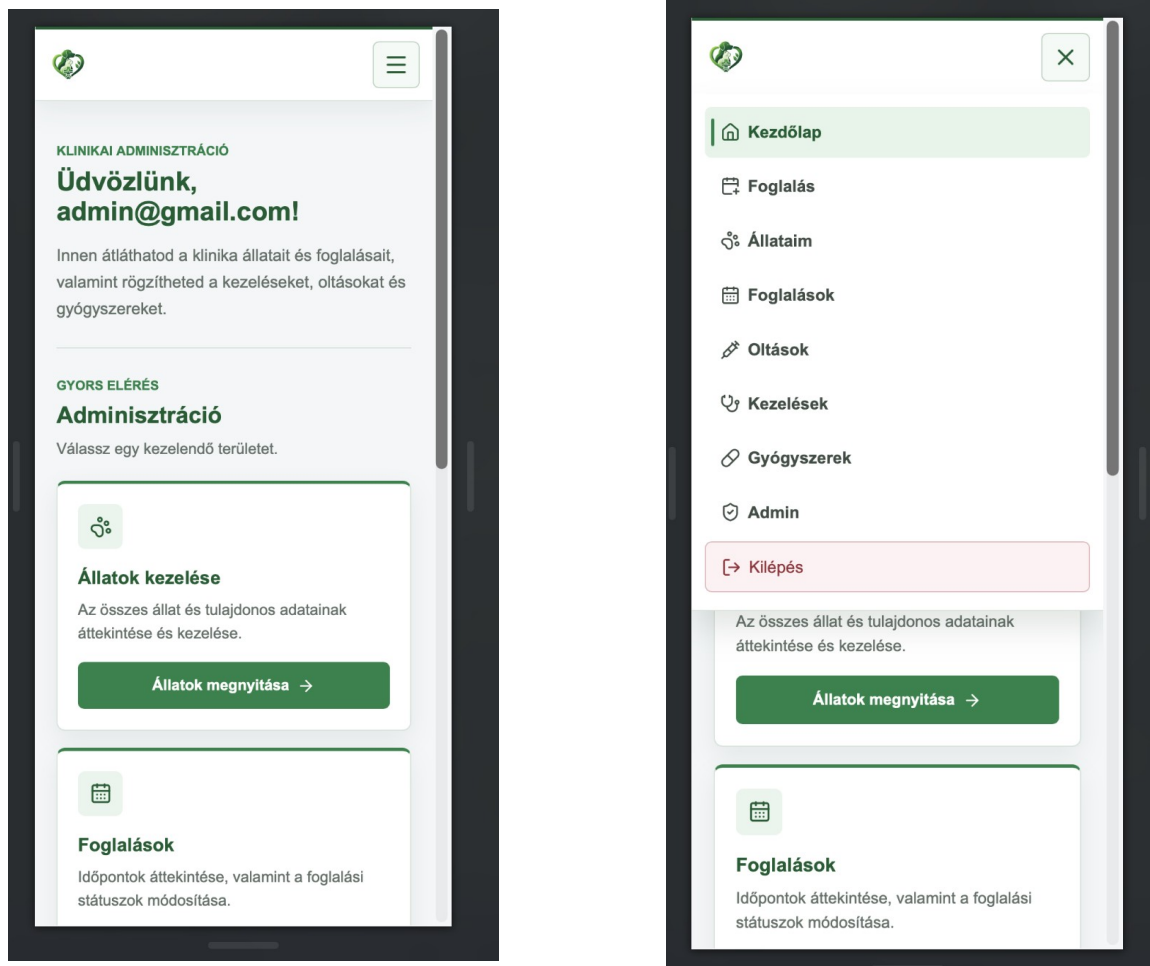


Figure 11 - Responsive mobile view and mobile navigation of the application

12. Using the Application

12.1 Login and Home Page

Using the system begins with login. After successful authentication, the user receives a home page appropriate to their permissions. A regular user can access their own pets and medical data, while an administrator can use the clinic's more complete management interface.

12.2 Managing Pets and Appointments

On the My Pets page, pet data can be reviewed and, depending on available permissions, a new pet can also be added. The Booking menu item is used to create a new appointment, while the Appointments page is used to review appointments already recorded. The administrator interface also supports changing statuses.

12.3 Vaccinations, Treatments and Medications

The Vaccinations, Treatments and Medications menu items display pets' medical history in separate categories. With administrator privileges, new treatment data, vaccination events or prescriptions can be recorded on the Medical Administration page.

12.4 User Flow and Permissions

The application can also be used by a regular pet owner. After registration and login, the user can add their own pet, view data linked to that pet and request an appointment. A newly created appointment is stored with pending status. An administrator can then review the booking and change its status. Regular users cannot modify another user's pets or bookings, and they cannot change booking statuses themselves.

12.5 Logout

The Logout button ends the user session. The client deletes the stored access and refresh tokens and then redirects the user to the login page.

Module	Backend relation	Purpose
Vaccinations	Vaccine, PetVaccination	Recording vaccine types and administration events
Treatments	MedicalRecord	Recording diagnoses and treatments
Medications	Prescription	Managing prescriptions

13. Installation and Running Guide

The system can be run locally on both Windows and macOS. The Windows-based startup process is presented as the primary examination environment, followed by the macOS version. The backend and frontend run in two separate terminal windows.

13.1 Required Software

Git, Python 3 and pip, Node.js and npm, and a modern browser are required. Visual Studio Code can be used to edit the source code. Package versions are defined by requirements.txt and react_frontend/package.json.

13.2 Downloading the Project

The project can be cloned from a Git repository and then opened from the petmedicalsysteM directory. The root folder contains separate backend and react_frontend directories.

```
git clone <repository-url>
cd petmedicalsysteM
```

13.3 Starting the Backend on Windows

The following commands can be run in PowerShell or Command Prompt. The virtual environment isolates the Python packages required by the project from other Python installations on the computer.

```
py -m venv venv
venv\Scripts\activate
py -m pip install -r requirements.txt
cd backend
py manage.py migrate
py manage.py runserver
```

After successful startup, the backend runs by default at `http://127.0.0.1:8000`. REST endpoints are available under the `/api/` path. If the root path is not separately defined, a 404 page displayed at the main address does not mean that the server is not running.

13.4 Starting the Frontend on Windows

In a new PowerShell or Command Prompt window, navigate to the project's frontend directory, install the JavaScript dependencies, and then start the Vite development server.

```
cd petmedicalsistem\react_frontend
npm install
npm run dev
```

The terminal displays the local frontend address. With Vite this is typically `http://localhost:5173`, but the current address shown in the terminal should always be opened in the browser.

13.5 Running on macOS

On macOS the process is the same, but the commands for Python and virtual-environment activation differ. For the backend, the following commands can be used from the project root directory:

```
python3 -m venv venv
source venv/bin/activate
python3 -m pip install -r requirements.txt
cd backend
python3 manage.py migrate
python3 manage.py runserver
```

On macOS, the frontend is started in the same way with `npm install` followed by `npm run dev` in the `react_frontend` directory.

13.6 Stopping the System

The development servers can be stopped in the terminal using `Ctrl+C`. When restarting, dependencies do not need to be installed every time; it is sufficient to activate the virtual environment and then start the backend and frontend.

14. Test Documentation

Testing of the final project includes functional, authorization, data-management and user-interface checks. The test cases examine functions actually implemented in the system.

ID	Test	Expected result
T01	Valid login	Access and refresh tokens are

		created
T02	Protected API without token	Access denied
T03	Create owner	New Owner record is created
T04	Assign pet to owner	Relationship is saved
T05	Upload pet image	Image is saved and can be displayed
T06	Create appointment	Appointment record is created
T07	Change appointment status	New status is retained
T08	Record vaccination event	PetVaccination is saved
T09	Record treatment history	MedicalRecord is created
T10	Record prescription	Prescription is created

14.1 User Interface Testing

Area	Check
Navigation	Every menu item leads to the correct page.
Booking	Required fields can be selected and the booking can be submitted.
My Pets	A new pet can be added and existing pets are displayed.
Appointments	Table data and statuses are readable.
Logout	The session is ended.
Error handling	Clear feedback is displayed for invalid or missing data.

14.2 API Testing

API endpoints can be tested through the browsable DRF interface or with an API client. Permission checks are particularly important: protected endpoints must not provide access to clinical data without a token.

14.3 Test Data

Demonstration users and clinical data can be used for testing. Separate test users can be used to verify the difference between regular-user and administrator permissions.

15. Possible Future Improvements

The current system provides a good foundation for a more complex clinical application. The following improvements could be logical next steps, but they are not part of the current version.

- Automatic notifications: email or in-application notifications when an appointment status changes, for example when it is accepted or cancelled.
- User profile management: allowing registered users to update their own profile information, such as name, email address and contact details.
- Refinement of role-based permissions with separate administrator, receptionist and veterinarian roles.
- Automatic appointment reminders by email or within the application.
- Automatic monitoring and notification of expiring vaccinations.
- Detailed search and filtering by owner, animal, date and status.
- Export of treatment sheets or prescriptions in PDF format.
- Use of a PostgreSQL database in a more production-like, multi-user environment.
- Automated backend and frontend tests.
- Further refinement of the responsive mobile view.
- Audit log for important administrative changes.

15.1 Possible Frontend Improvements

Future frontend development could introduce a unified notification system, more detailed form validation, automated component and user-interface tests, and further mobile-view refinements.

16. Summary

The system manages owners, pets, veterinarians, appointments, medical records, vaccines, vaccination events and prescriptions. JWT-based authentication and permission-aware query filtering protect clinical data. In addition to administrative use, registered pet owners can manage their own pets and request appointments, while booking status changes remain under administrator control.

The system manages owners, pets, veterinarians, appointments, medical records, vaccines, vaccination events and prescriptions. JWT-based authentication protects the API endpoints, while Django Admin provides a separate administration option.

The user interface presents clinical tasks in clearly separated menu items. The React-based client, Django REST API and JWT authentication together form a complete multi-user web application that can be run in both Windows and macOS development environments.

References

<https://www.w3schools.com/>

<https://react.dev/learn>

<https://www.djangoproject.com/start/>